

FIAP GRADUAÇÃO

TECNOLOGIA EM DESENVOLVIMENTO DE SISTEMAS

DevOps Tools & Cloud Computing

Exercício de Dockerfile

PROF. João Menk

profjoao.menk@fiap.com.br

Este exercício tem como objetivo praticar a criação de um Dockerfile para rodar um aplicativo em Java Spring Boot

O Docker é uma ferramenta que permite empacotar uma aplicação junto com suas dependências e configurações em um Container, proporcionando assim uma maior portabilidade e facilidade na implantação de aplicações em diferentes ambientes

O Java Spring Boot é um framework de desenvolvimento Web que fornece uma plataforma robusta para a criação de Aplicativos Web escaláveis e de alto desempenho

Combinar essas duas ferramentas nos permite criar um ambiente de desenvolvimento eficiente e altamente portátil para aplicativos Java

Criando um App

Primeiramente crie seu App utilizando o **Spring Initializr**

Acesse em seu navegador: **<https://start.spring.io/>**

1) Usaremos as seguintes configurações:

Project: **Maven**
Language: **Java**
Spring Boot: **4.0.6**
Group: **com.example**
Artifact: **app**
Name: **app**
Packaging: **Jar**
Java: **17**

2) Clique no botão "Add Dependencies" e adicione a seguinte dependência:
Spring Web

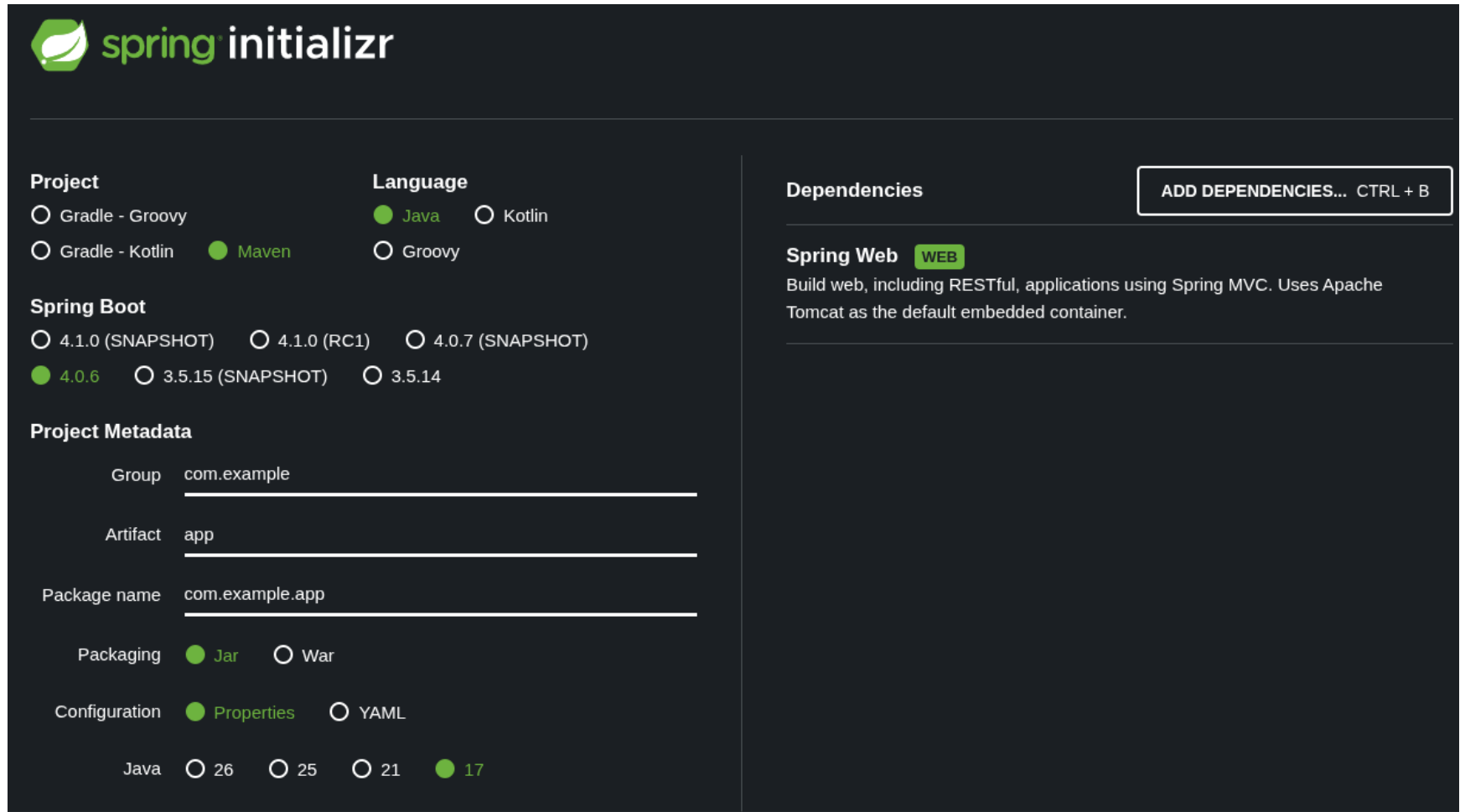
Para criar um aplicativo Web MVC

- 3) Clique no botão "Generate" para baixar um arquivo ZIP contendo o projeto Spring Boot
- 4) Depois de baixar e extrair o arquivo ZIP, você pode abrir o projeto em sua IDE favorita e começar a implementar sua aplicação Spring Boot
- 5) Fique a vontade para realizar qualquer tipo de codificação nesse App Java Spring Boot. Aqui iremos passar uma página simples, pois o foco é a criação do Dockerfile e o Container com o App funcionando
- 6) Altere o código gerado em:
`\src\main\java\com\example\app\AppApplication.java`

O professor irá passar o fonte desse arquivo

Criando um App

Tela demonstrando as propriedades do App a ser gerado



The image shows the Spring Initializr web application interface. It is a dark-themed form for configuring a new Spring project. The interface is divided into several sections: Project, Language, Spring Boot, Project Metadata, and Dependencies. The Project section has radio buttons for Gradle - Groovy, Gradle - Kotlin, and Maven (selected). The Language section has radio buttons for Java (selected), Kotlin, and Groovy. The Spring Boot section has radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (RC1), 4.0.7 (SNAPSHOT), 4.0.6 (selected), 3.5.15 (SNAPSHOT), and 3.5.14. The Project Metadata section has input fields for Group (com.example), Artifact (app), and Package name (com.example.app). The Packaging section has radio buttons for Jar (selected) and War. The Configuration section has radio buttons for Properties (selected) and YAML. The Dependencies section has a button to add dependencies and a list of selected dependencies, including Spring Web (WEB). The Spring Web dependency is described as 'Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.'

spring initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT)

☒ 4.0.6

☐ 3.5.15 (SNAPSHOT)

☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies [ADD DEPENDENCIES... CTRL + B](#)

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

\src\main\java\com\example\app\AppApplication.java

```
package com.example.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class AppApplication {
    public static void main(String[] args) {
        SpringApplication.run(AppApplication.class, args);
    }

    @GetMapping("/")
    public String greeting() {
        return "Saudações do Java!";
    }
}
```

Para copiar o projeto de sua maquina para a VM

1) Realize o Upload do **app.zip** para o Cloud Shell

2) Descompacte o projeto com o comando (ainda no Cloud Shell):

```
unzip app.zip
```

3) Copie o diretório criado **app** recursivamente para sua VM com o comando:

```
scp -r app <user>@<IP_VM>:/home/<user>/
```

Exemplo:

```
scp -r app admlnx@20.197.225.21:/home/admlnx/
```

4) Se conecte em sua VM e continue a tarefa

1) Crie o Dockerfile (dentro do seu Projeto), a imagem e o Container para rodar esse App

O Dockerfile deve conter as seguintes tarefas:

1 - Imagem a ser utilizada como base: **maven:3.9.9-eclipse-temurin-17-alpine**

2 - Atualize o repositório do Alpine (APK)

3 - Crie um novo usuário com o nome: **userapp**

4 - Use esse usuário para executar os próximos comandos

5 - Defina um diretório padrão como **/app**

6 – Copie todos os arquivos do seu projeto para o diretório padrão

7 - Realize o Build da sua Aplicação com o Maven

8 - Exponha a porta **8080**

9 - Execute o comando: **java -jar app-0.0.1-SNAPSHOT.jar** (Utilize CMD)

- 2) Crie a imagem **meu-app-docker** a partir do seu Dockerfile criado
- 3) Rode o Container em segundo plano com o nome **myapp**
- 4) Realize os testes em **<http://localhost:8080>**
- 5) Entre no terminal do Container através do comando **docker container exec** e verifique os diretórios, usuário criado, arquivo copiado etc (*sh*)
- 6) Verifique os LOGs do Container pelo terminal

LIMPAR O LABORATÓRIO DO DOCKER COMPOSE

`docker container rm -f myapp`

`docker image rm meu-app-docker`



Copyright © 2026 Prof. João Menk

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).